

MODUL PRAKTIK/PRAKTIKUM

IF451 – Advance Web Programming

PROGRAM SARJANA S1

FAKULTAS TEKNIK DAN INFORMATIKA

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK DAN INFORMATIKA
UNIVERSITAS MULTIMEDIA NUSANTARA

Gedung B Lantai 5, Kampus UMN

Jl. Scientia Boulevard, Gading Serpong, Tangerang, Banten-15811 Indonesia

Telp: +62-21.5422.0808, web: <http://www.umn.ac.id>



Modul Week 6 - Connecting Node.js to MySQL Database

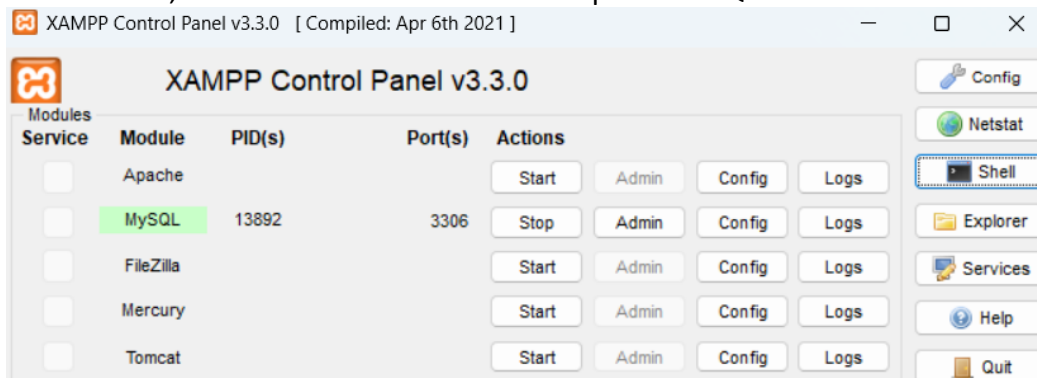
I. Koneksi Mysql ke NodeJS server

Pada week ini, kita akan menghubungkan database “playlist” yang sudah dibuat sebelumnya pada latihan week 5 ke NodeJS. Koneksi ini dapat dilakukan melalui module driver menjadi antarmuka (API) antara NodeJS dengan MySQL database sehingga NodeJS dapat melakukan query ke dalam MySQL. Driver yang akan kita gunakan adalah **MySQL2**.

II. Membuat Koneksi antara MySQL database dengan NodeJS

A. Setup XAMPP

1. Buka XAMPP, kemudian tekan button “Start” pada MYSQL.



Data yang tersimpan (melalui phpmyAdmin & harus menyalakan Apache):

ID	Title	Artist	Album	Duration
1	Blinding Lights	The Weeknd	After Hours	00:03:20
2	Levitating	Dua Lipa	Future Nostalgia	00:03:23
3	Shape of You	Ed Sheeran	÷ (Divide)	03:53:00

2. Buat Directory Baru bernama **Week6_NIM**.
3. Jalankan **npm init -y** untuk membuat package.json yang berisi dependencies akan modul yang akan digunakan lalu jalankan **npm install mysql2** pada terminal untuk menginstal driver MySQL pada NodeJS.

4. Buat file baru dengan nama **koneksi.js** yang berisi informasi antar MySQL database dan NodeJS. Ubah value pada database dengan menyesuaikan nama database pada mysql

```
JS koneksi.js > [❌] conn
1  const mysql = require("mysql2");
2  
3  const conn = mysql.createConnection({
4      host: 'localhost',
5      user: 'root',
6      password: '',
7      database: 'welliam_playlist_cli'
8  });
9
10 conn.connect(err => {
11     if (err) {
12         console.log(err);
13     } else {
14         console.log("Connected to MySQL!");
15     }
16 });
17
18 module.exports = conn;
```

5. Ubah html yang sudah dibuat pada week 4 untuk menyesuaikan dengan playlist lagu.

```

index.html > ...
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Database Connection</title>
7 </head>
8 <body>
9   <h1>Playlist Lagu</h1>
10  <form action="/create" method="POST">
11    <label for="Title">Title </label>
12    <input type="text" name="Title" placeholder="Input Title here" required >
13    <br>
14
15    <label for="Artist">Artist</label>
16    <input type="number" name="Artist" placeholder="Input Artist here">
17    <br>
18
19    <label for="Album">Album </label>
20    <input type="text" name="Album" placeholder="Input Album here">
21    <br>
22
23    <label for="Duration">Duration </label>
24    <input type="text" name="Duration" placeholder="Input Duration (ex: 3:45)">
25    <br>
26    <br>
27
28    <button type="submit">Save</button>
29  </form>
30
31  <!-- TABEL -->
32 </body>
33 </html>

```

6. Buat file baru dengan nama **app.js** yang menjalankan `.require("koneksi.js")` untuk meng-importnya ke dalam `app.js`

```

1 const http = require('http');
2 const fs = require('fs');
3 const conn = require('./koneksi.js');
4 const querystring = require('querystring');

```

7. Untuk menampilkan semua data pada table, kita dapat menjalankan query "SELECT * FROM [table_name]", lalu simpan ke dalam sebuah variabel bernama **q**. Query kemudian dapat dimasukkan ke dalam function `.query()` pada variabel **conn** (object) yang kemudian akan menjalankan query tersebut pada database. Setelah data ditemukan, setiap elemen **result** akan di tampilkan sebagai table data.

```

6  const server = http.createServer((req, res) => {
7    if (req.method === 'GET' && req.url === '/') {
8      fs.readFile("index.html", 'utf-8', (err, content) => {
9        if (err) {
10         res.writeHead(500, { 'Content-Type': 'text/plain' });
11         res.end('Error Loading file');
12         return;
13       }
14
15       const q = "SELECT * FROM Playlist";
16       conn.query(q, (err, results) => {
17         if (err) {
18           console.log(err);
19           res.writeHead(500, { 'Content-Type': 'text/plain' });
20           return res.end('Database error');
21         }
22         const rows = results.map(elm => `
23         <tr>
24           <td>${elm.ID}</td>
25           <td>${elm.Title}</td>
26           <td>${elm.Artist}</td>
27           <td>${elm.Album}</td>
28           <td>${elm.Duration}</td>
29         </tr>
30         `).join('');
31
32         const html = content.replace(`<!-- TABEL -->`, `
33         <h2>Daftar Data</h2>
34         <table border="1" cellPadding="5">
35           <tr>
36             <th>ID</th>
37             <th>Title</th>
38             <th>Artist</th>
39             <th>Album</th>
40             <th>Duration</th>
41           </tr>
42           ${rows}
43         </table>
44         `);
45         res.writeHead(200, { 'Content-Type': 'text/html' });
46         res.end(html);
47       });
48     }
49   });
50 }

```

8. Untuk menambah data, kita dapat mengirimkan post request, kemudian proses request tersebut dengan memasukkan valuesnya pada query "INSERT (...) VALUES (?, ?, ?, ?).
9. ? Pada query insert merupakan metode untuk menerapkan **parameterized queries**. Parameterized queries merupakan cara untuk menyisipkan value secara aman dengan menyiapkan **prepared** statements. Query tersebut dapat menghindari terjadinya serangan SQL injection dimana input akan terbaca menjadi **string literal**, bukan **executable SQL**.

Misalnya untuk searching input : 1" "OR 1=1

Tanpa placeholder:

```
SELECT * FROM Playlist WHERE id = 1 OR 1=1;
```

Menggunakan placeholder:

```
SELECT * FROM Playlist WHERE id = '1 OR 1=1';
```

Sehingga server tidak akan memunculkan semua data secara tidak sengaja.

Placeholder kemudian dapat diisi sebagai argumen kedua pada method .query(q, ...).

```
52 // CREATE
53 else if (req.method === 'POST' && req.url === '/create') {
54   let body = '';
55
56   // kumpulkan semua data
57   req.on('data', chunk => { body += chunk.toString(); });
58
59   req.on('end', () => {
60     const form = querystring.parse(body);
61
62     const q = 'INSERT INTO Playlist (Title, Artist, Album, Duration) VALUES (?, ?, ?, ?)';
63     conn.query(q, [form.Title, form.Artist, form.Album, form.Duration], (err, result) => {
64       if (err) {
65         console.error('Insert failed:', err);
66         res.writeHead(500, { 'Content-Type': 'text/plain' });
67         return res.end('Database insert error');
68       }
69       console.error('Insert Success');
70       res.writeHead(302, { Location: '/' });
71       res.end();
72     });
73   });
74 }
75
76 };
```

10. Tambahkan server.listen lalu jalankan dengan **nodemon app.js**.

```
79 server.listen(3000, () => {
80   console.log('server berjalan di http://localhost:3000');
81 });
```

11. Perhatikan hasilnya dan lakukan insertion. Cek kembali database untuk memeriksa apakah data masuk atau tidak.

Playlist Lagu

Title

Artist

Album

Duration

Daftar Data

ID	Title	Artist	Album	Duration
1	Blinding Lights	The Weeknd	After Hours	00:03:20
2	Levitating	Dua Lipa	Future Nostalgia	00:03:23
3	Shape of You	Ed Sheeran	÷ (Divide)	03:53:00

Tugas

Ubah koneksi untuk menyesuaikan dengan tugas pada week 5 sebelumnya dan sesuaikan input dan tabel dengan table yang telah dibuat pada tugas tersebut!

Screenshot/screen record setiap hasil pengerjaan

1. Tampilan website (sebelum & sesudah input)
2. Tampilan database (sebelum & sesudah input)

Kumpulkan semua file dalam .zip dengan penamaan (Nama_NIM_TugasWeek6).